Threshold MLSAG key generation and signing

Abstract

This note abstracts the threshold MLSAG implementation in `sl-mpc-mlsag`. Distributed key generation is the Feldman–Shamir commit–open of Protocol 9 in [1], instantiated on the Ristretto curve, followed by one extra round in which every party contributes to the MobileCoin key image [2]. Distributed signing spends one output whose one-time key is the DKG key shifted by a public tweak. It first commits to a fresh per-party session id and to the spend nonce, opens that nonce under the combined session id together with each party's share of the output's key image, and then closes only the spend response of a MobileCoin `RingMLSAG`. Whoever already knows the blinding difference fills the decoy responses and the amount-layer response. The threshold parties never see the blinding difference.

1 Group, parties, and hashes

Let $\mathbb{G}$ be the Ristretto group, written additively, with generator G and scalar field $\mathbb{F}$. Identity is denoted O. There are n parties $P_0, \ldots, P_{n-1}$ and a threshold $2 \le t \le n$. Party P_i is evaluated at the Shamir point $\omega_i = i + 1 \in \mathbb{F}$. For a set S of party indices and $i \in S$,

$$\lambda_i(S) = \prod_{\substack{j \in S \\ j \neq i}} \frac{\omega_j}{\omega_j - \omega_i}.$$

If $(d_i)_{i \in S}$ are Shamir shares of a secret x on those points, then $x = \sum_{i \in S} \lambda_i(S)\, d_i$.

Hash to point. For a public key $P \in \mathbb{G}$,

$$H_p(P) = \mathsf{from_hash}\big(\text{Blake2b-512}(\texttt{mc_onetime_key_hash_to_point} \,\|\, \mathsf{compress}(P))\big).$$

The key image of a secret x with $P = xG$ is $I = x \cdot H_p(P)$.

Ring challenge. A ring has M members. For a message m, a compressed key image I, and points L_0, R_0, L_1,

$$H_n(m, I, L_0, R_0, L_1) = \mathsf{from_hash}\big(\text{Blake2b-512}(\texttt{mc_ring_mlsag_challenge} \,\|\, m \,\|\, I \,\|\, \mathsf{compress}(L_0) \,\|\, \mathsf{compress}(R_0) \,\|\, \mathsf{c}$$

Discrete-log proof. A proof of knowledge of w such that $Y = wG$, bound to a 32-byte session identifier sid, is a pair (T, s): sample $r \in \mathbb{F}$, set $T = rG$,

$$c = \mathsf{SHA256}\big(\texttt{DLogProof-Challenge} \,\|\, \mathsf{sid} \,\|\, \mathtt{y} \,\|\, \mathsf{compress}(Y) \,\|\, \mathtt{t} \,\|\, \mathsf{compress}(T) \,\|\, \texttt{base-point} \,\|\, \mathsf{compress}(G)\big)$$

reduced into $\mathbb{F}$ by `from_bytes_mod_order`, and $s = r + cw$. Verification recomputes c from (Y, T) and checks $sG = T + cY$.

DH-tuple proof. A proof that (G, Q, A, B) is a DH tuple with witness w, that is $A = wG$ and $B = wQ$, is a pair (e, z). The prover samples r and sets $X = rG$ and $Y = rQ$. The challenge is one SHA-256 transcript. It begins with the label `ZK-DH-Tuple-Proof`. It then absorbs, in order, the label `session_id` with sid, the label `aux` with the auxiliary string, and the labels `G`, `Q`, `A`, `B`, `X`, `Y`, each followed by that point compressed. Every absorption is length-prefixed: the label length as eight little-endian bytes, the label, the message length as eight little-endian bytes, and the message. The digest is read after appending the length of `ZK-DH-Challenge` as four little-endian bytes, that label, and the output length 32 as four little-endian bytes. Write $d \in \{0,1\}^{256}$ for that digest.

$$e = \mathsf{from_bytes_mod_order}(d).$$

This reads d as a little-endian integer and reduces it modulo the Ristretto scalar order $\ell = 2^{252} + 27742317777372353535851937790883648493$. In the code the call is `Scalar::reduce_from_bytes`, and for a Ristretto scalar that function is `from_bytes_mod_order`. The proof outputs $z = r + ew$. The verifier rebuilds $X = zG - eA$ and $Y = zQ - eB$ inside the same transcript and accepts when the recomputed challenge equals e. All four points G, Q, A, B must be nonzero.

2 How each round is delivered

The host delivers every protocol message on one of two channels. **Signed** means a transport signature by the sender, checked by every recipient before the payload is parsed. That signature is not the discrete-log proof or the DH-tuple proof inside the payload. **Encrypted** means the host encrypts the payload to one recipient. A broadcast payload is identical for every recipient, so it is signed and left in the clear.

Protocol	Round	Payload	Channel
DKG	1	$\mathsf{Msg1}_i$	Broadcast, signed only.
DKG	2	$\mathsf{Msg2}_{i\to j}$	P2P, encrypted to P_j and signed by P_i.
DKG	3	$\mathsf{Msg3}_i$	Broadcast, signed only.
DSG	1	$\mathsf{Msg1}_j$	Broadcast to S, signed only.
DSG	2	$\mathsf{Msg2}_j$	Broadcast to S, signed only.
DSG	ring	$\mathcal{T}$	Broadcast to S, signed only.
DSG	3	$\mathsf{Msg3}_j$	Broadcast to S, signed only.

Only DKG round 2 is peer-to-peer. It carries the Shamir payload $\sigma_{i\to j}$, which P_j alone may read. Every other round is one signed broadcast: every party in the receiving set must see the same bytes, and none of those bytes is a per-recipient secret. The implementation does not sign or encrypt these payloads. The words above say what the host does with them. In signing, every party in S must see the same round-1, round-2, ring, and round-3 bytes. The ring transcript leaves the spend response at index π equal to 0, so it shows π to anyone who receives it.

3 Distributed key generation

DKG has two phases. The first is Feldman verifiable secret sharing with a commit–open, and it must be run by all n parties. The second assembles $I = x \cdot H_p(P)$. The implementation always

runs that second phase on the full set $\{0, \ldots, n-1\}$, even when $t < n$. A later signature may be produced by any subset of size at least t, but the key image itself is an n-party output.

Protocol MLSAG-DKG

Parties. $P_0, \ldots, P_{n-1}$.
Public input. n and t.
Private input. Each P_i has fresh randomness.

Phase A — Feldman–Shamir commit–open.

Local sampling. Each P_i samples a session identifier $\mathsf{sid}_i \in \{0,1\}^{256}$, a blinding $r_i \in \{0,1\}^{256}$, a chain-code seed $\mathsf{cc}_i \in \{0,1\}^{256}$, and a polynomial $f_i \in \mathbb{F}[X]$ of degree at most $t-1$. Write $f_i(X) = \sum_{k=0}^{t-1} a_{i,k} X^k$ and $A_{i,k} = a_{i,k} G$, so the Feldman vector is $\mathbf{A}_i = (A_{i,0}, \ldots, A_{i,t-1})$. For every receiver P_j, prepare the 64-byte payload

$$\sigma_{i \to j} = \mathsf{encode}(f_i(\omega_j)) \,\|\, \mathsf{cc}_i.$$

The round-1 commitment does *not* cover these payloads:

$$C_i = \mathsf{SHA256}\big(\texttt{SL-Keygen-Commitment} \,\|\, \mathsf{sid}_i \,\|\, \mathsf{be}(i) \,\|\, \mathsf{compress}(\mathbf{A}_i) \,\|\, r_i \,\|\, \mathsf{cc}_i\big).$$

Round 1. *Channel: broadcast, signed only.* Each P_i **broadcast**

$$\mathsf{Msg1}_i = (\, i,\ \mathsf{sid}_i,\ C_i \,).$$

Round 2. *Channel: peer-to-peer, encrypted to the recipient and signed by the sender.* Each P_i, on a vector of n round-1 messages whose senders are exactly $\{0, \ldots, n-1\}$ and whose own entry matches the (sid_i, C_i) it sent, computes

$$\mathsf{sid} = \mathsf{SHA256}\Big(\|_{i=0}^{n-1} \big(\mathsf{be32}(i) \,\|\, \mathsf{sid}_i\big)\Big),$$

with the pairs sorted by party index. It then proves knowledge of every coefficient of f_i. The proof session identifier is

$$\mathsf{sid}_i^{\mathrm{dl}} = \mathsf{SHA256}\big(\texttt{SL-EDDSA-DLOG-PROOF} \,\|\, \mathsf{sid} \,\|\, \mathsf{be32}(i) \,\|\, \texttt{DLOG-PROOF-1-SID}\big),$$

and $\pi_{i,k}$ is a discrete-log proof for $A_{i,k} = a_{i,k} G$ under $\mathsf{sid}_i^{\mathrm{dl}}$. For each recipient P_j, party P_i **send**s privately

$$\begin{aligned}\mathsf{Msg2}_{i \to j} = \big(&i,\ j,\ \mathsf{sid},\ r_i,\ \mathsf{cc}_i,\\ &\mathbf{A}_i,\ \sigma_{i \to j},\ (\pi_{i,0}, \ldots, \pi_{i,t-1})\big).\end{aligned}$$

Only $\sigma_{i \to j}$ depends on j. The other fields are identical in every copy. The message type carries $\sigma_{i \to j}$ in the clear. The host encrypts the whole $\mathsf{Msg2}_{i \to j}$ to P_j and attaches P_i's transport signature. P_j is the only reader.

Local opening. P_i accepts a batch of n round-2 messages if and only if every message is addressed to i, carries sid, opens the round-1 commitment of its sender, the t discrete-log proofs verify against $\mathbf{A}_j$, the chain-code half of $\sigma_{j \to i}$ equals the opened cc_j, and the Feldman check holds:

$$f_j(\omega_i)\, G = \sum_{k=0}^{t-1} \omega_i^k A_{j,k}.$$

It then sets

$$d_i = \sum_{j=0}^{n-1} f_j(\omega_i), \qquad \mathbf{A} = \sum_{j=0}^{n-1} \mathbf{A}_j, \qquad P = A_0,$$

and aborts unless $d_i G = \mathbf{A}(\omega_i)$. The stored public share of party P_j is the full-quorum additive share

$$Y_j = \lambda_j(\{0, \ldots, n-1\})\, \mathbf{A}(\omega_j),$$

so $\sum_j Y_j = P$. Auxiliary outputs, unused by the MLSAG spend itself, are

$$\mathsf{cc} = \mathsf{SHA256}\big(\texttt{SL-Keygen-ChainCode} \,\|\, \mathsf{cc}_0 \,\|\, \cdots \,\|\, \mathsf{cc}_{n-1}\big)$$

and $\mathsf{key_id} = \mathsf{SHA256}(\mathsf{compress}(P))$.

Phase B — key image. Every party sets $Q = H_p(P)$ and

$$\mathsf{sid}_I = \mathsf{SHA256}\big(\texttt{SL-MLSAG-KEYIMAGE} \,\|\, \mathsf{sid} \,\|\, 0 \,\|\, 1 \,\|\, \cdots \,\|\, n-1\big).$$

Round 3. *Channel: broadcast, signed only.* Each P_i forms its full-quorum additive share $k_i = \lambda_i(\{0, \ldots, n-1\})\, d_i$, the points $K_i = k_i G$ and $I_i = k_i Q$, and a DH-tuple proof π_i for (G, Q, K_i, I_i) under sid_I with auxiliary string $\mathsf{be32}(i)$. It **broadcast**

$$\mathsf{Msg3}_i = \big(i,\ \mathsf{sid}_I,\ \mathsf{compress}(I_i),\ \pi_i\big).$$

K_i is not sent. Every party recomputes it as Y_i.

Local assembly. P_i accepts the n round-3 messages when the sender set is exactly $\{0, \ldots, n-1\}$, every session identifier equals sid_I, every I_j is nonzero, and π_j verifies on (G, Q, Y_j, I_j). It sets $I = \sum_j I_j$ and aborts if $I = O$.

Output. Each P_i stores an MLSAG key share

$$\big(t,\ n,\ i,\ d_i,\ P,\ (Y_j)_{j<n},\ \mathsf{key_id},\ \mathsf{cc},\ \mathsf{sid},\ \mathsf{sid}_I,\ I,\ \{0, \ldots, n-1\}\big).$$

Honest parties agree on P and I, and $I = x \cdot H_p(P)$ for the secret $x = \sum_i a_{i,0} = \sum_i k_i$.

---

4 Distributed signing

The DKG key is the root spend key. In this section write b for its secret, $B = bG$ for the public key that Section 3 calls P, and $I_B = b \cdot H_p(B)$ for the DKG key image. DKG runs once. Each output is spent with its own one-time key

$$P = B + \tau G, \qquad x = b + \tau,$$

where $\tau \in \mathbb{F}$ is a public per-output tweak. (It is written τ here because t is the threshold.) Whoever holds the view key a computes it; for MobileCoin it is $\tau = \mathrm{H_s}(a \,\|\, i) + \mathrm{H_s}(aR)$ for subaddress i and output public key R. The signers take τ as given. They check that $B + \tau G$ is the real ring member and bind τ into the session identifier, but they cannot check how τ was derived. The key image of the output, $I = x \cdot H_p(P)$, is computed during signing. With $\tau = 0$ the output key is B and the key image is I_B.

Signing uses any set S of at least t parties from the DKG. Rounds 1 and 2 commit and open a spend nonce, fix one session identifier, and assemble the key image I. A party that knows the blinding difference then broadcasts the rest of the ring. Round 3 closes the spend response. The signers never see that blinding difference.

Protocol MLSAG-DSG

Parties. $\{P_j : j \in S\}$, with $|S| \geq t$ and $S \subseteq \{0, \ldots, n-1\}$.
Public input. The DKG output B, $(Y_j)_{j<n}$, and the DKG session identifier sid_K. A canonical tweak $\tau \in \mathbb{F}$. A message m. A ring $(P_i, C_i)_{i<M}$ and an index $\pi < M$ with $P_\pi = B + \tau G$, both B and P_π nonzero, and every $P_i \neq O$. An output commitment C_{out}.
Private input. Each P_j has its Shamir share d_j of b. One party has $z \in \mathbb{F}$ with $C_{\text{out}} - C_\pi = zG$.

Local sampling. Each P_j sets $P = B + \tau G$, $Q = H_p(P)$ and

$$k_j = \lambda_j(S)\,(d_j + \tau).$$

Because $\sum_{j \in S} \lambda_j(S) = 1$, the values $d_j + \tau$ are Shamir shares of $x = b + \tau$ and $\sum_{j \in S} k_j = x$. For every $\ell \in S$ it also forms the public counterpart of k_ℓ from the stored DKG public shares:

$$K_\ell = Y_\ell \cdot \frac{\lambda_\ell(S)}{\lambda_\ell(\{0, \ldots, n-1\})} + \lambda_\ell(S)\,\tau\,G = k_\ell G.$$

It samples $\mathsf{sid}_j \in \{0,1\}^{256}$, a blind $\beta_j \in \{0,1\}^{256}$, two seeds seed_j and seed_j^I, and $\alpha_{0,j} \in \mathbb{F}$. It sets $L_{0,j} = \alpha_{0,j} G$ and $R_{0,j} = \alpha_{0,j} Q$. The commitment does not include a proof:

$$\mathsf{com}_j = \mathsf{SHA256}\big(\mathsf{sid}_j \,\|\, \mathsf{be32}(j) \,\|\, \mathsf{compress}(L_{0,j}) \,\|\, \mathsf{compress}(R_{0,j}) \,\|\, \beta_j\big).$$

Every party also forms the same public binding bind. Its label is `SL-MLSAG-SIGN`. The bytes are sid_K, $\mathsf{compress}(B)$, $\mathsf{encode}(\tau)$, $\mathsf{compress}(P)$, one byte $|S|$, the indices in S in increasing order, π, $|m|$, m, $\mathsf{compress}(C_{\text{out}})$, M, then each $\mathsf{compress}(P_i) \,\|\, \mathsf{compress}(C_i)$. Here $\mathsf{encode}(\tau)$ is the canonical 32-byte little-endian encoding of τ. Lengths π, $|m|$, and M are eight-byte little-endian integers. bind is not a message field. Since τ and P are fixed in bind before any nonce is sampled, a signing session is tied to one output.

Round 1. *Channel: broadcast, signed only.* Each P_j **broadcast**

$$\mathsf{Msg1}_j = (\, j,\ \mathsf{sid}_j,\ \mathsf{com}_j\,).$$

$L_{0,j}$, $R_{0,j}$, and β_j stay local.

Round 2. *Channel: broadcast, signed only.* Each P_j, on $|S|$ round-1 messages whose senders are exactly S and whose own entry matches the $(\mathsf{sid}_j, \mathsf{com}_j)$ it sent, sorts those messages by party index and sets

$$\mathsf{sid} = \mathsf{SHA256}\big(\mathsf{bind} \,\|\, \big\|_{j \in S} \mathsf{be32}(j) \,\|\, \big\|_{j \in S} \mathsf{sid}_j\big).$$

It proves the DH tuple $(G, Q, L_{0,j}, R_{0,j})$ with witness $\alpha_{0,j}$ under sid, auxiliary string $\mathsf{be32}(j)$, and proof coins taken from seed_j. Call that proof π_j. It also forms its key-image share $I_j = k_j Q$ and proves the DH tuple (G, Q, K_j, I_j) with witness k_j under sid and auxiliary string

$$\texttt{SL-MLSAG-SIGN-KEY-IMAGE} \,\|\, \mathsf{be32}(j).$$

The proof coins come from a ChaCha20 stream seeded with

$$\mathsf{SHA256}\big(\texttt{SL-MLSAG-SIGN-KEY-IMAGE-PROOF-SEED} \,\|\, \mathsf{seed}_j^I \,\|\, \mathsf{sid} \\ \|\, \mathsf{be32}(j) \,\|\, \mathsf{compress}(K_j) \,\|\, \mathsf{compress}(I_j)\big).$$

Call that proof π_j^I. It **broadcast**

$$\mathsf{Msg2}_j = \big(j,\ \mathsf{sid},\ \beta_j,\ \mathsf{compress}(L_{0,j}),\ \mathsf{compress}(R_{0,j}),\ \pi_j, \\ \mathsf{compress}(I_j),\ \pi_j^I\big).$$

K_j is not sent. Every party computes it from Y_j and τ.

Local opening. P_j accepts the $|S|$ round-2 messages when the sender set is exactly S, every session identifier equals sid, every $L_{0,\ell}$ and $R_{0,\ell}$ is nonzero, each opening hash equals the stored com_ℓ, π_ℓ verifies on $(G, Q, L_{0,\ell}, R_{0,\ell})$, every I_ℓ is nonzero, and π_ℓ^I verifies on (G, Q, K_ℓ, I_ℓ). It sets

$$L_0 = \sum_{\ell \in S} L_{0,\ell}, \qquad R_0 = \sum_{\ell \in S} R_{0,\ell}, \qquad I = \sum_{\ell \in S} I_\ell,$$

and aborts if $I = O$. Then $I = x \cdot Q = (b + \tau) \cdot H_p(P)$, the key image of the output. The shares I_ℓ need no round-1 commitment: each is fixed by K_ℓ, which the DKG determines, so no party can bias I.

Ring transcript.

Channel: broadcast, signed only. The party holding z reads (L_0, R_0, I) from the round-2 broadcast and aborts unless $C_{\text{out}} - C_\pi = zG$. For every $i \neq \pi$ it samples $r_{i,0}, r_{i,1} \in \mathbb{F}$ by reducing 64 uniform bytes, and samples α_1 the same way. It sets $L_1 = \alpha_1 G$ and $r_{\pi,0} = 0$, runs the challenge walk below starting at π, using (L_0, R_0, L_1) at index π, and then sets

$$r_{\pi,1} = \alpha_1 - c_\pi z.$$

It **broadcast**

$$\mathcal{T} = \big((r_{i,0}, r_{i,1})_{i<M},\ L_1\big).$$

Every signer must receive this same $\mathcal{T}$. The slot $r_{\pi,0}$ is still 0, so $\mathcal{T}$ shows π.

Round 3. *Channel: broadcast, signed only.* Each P_j, on that $\mathcal{T}$, recomputes the same walk and reads c_π. It **broadcast**

$$\mathsf{Msg3}_j = \big(j,\ \mathsf{sid},\ r_{\pi,0}^{(j)}\big), \qquad r_{\pi,0}^{(j)} = \alpha_{0,j} - c_\pi k_j.$$

Local assembly. P_j accepts the $|S|$ round-3 messages on the same $\mathcal{T}$ when the sender set is exactly S, every session identifier equals sid, and every response is a canonical scalar. For each sender ℓ it aborts unless $r_{\pi,0}^{(\ell)} G + c_\pi K_\ell = L_{0,\ell}$, with K_ℓ the tweaked public share from the local sampling. It writes $r_{\pi,0} = \sum_{\ell \in S} r_{\pi,0}^{(\ell)}$ into $\mathcal{T}$ and outputs

$$\big(c_0,\ (r_{i,0}, r_{i,1})_{i<M},\ I\big),$$

where c_0 is the challenge at index 0 on this completed ring.

---

5 Verification

The signature is $(c_0, (r_{i,0}, r_{i,1})_{i<M}, I)$. It does not contain the ring. The verifier is given the message m, the output commitment C_{out}, and the ring that was signed. Slot i of that ring supplies two points: P_i, the onetime public key, and C_i, the amount commitment of that output. Both were already published with the output. The real slot is one of them, and the others are decoys. The verifier uses every slot and is not given the real index or the tweak τ. Reject a non-canonical c_0 or response. For $i = 0, \ldots, M-1$,

$$\begin{aligned} L_0^{(i)} &= r_{i,0} G + c_i P_i, \\ R_0^{(i)} &= r_{i,0}\, H_p(P_i) + c_i I, \\ L_1^{(i)} &= r_{i,1} G + c_i \big(C_{\text{out}} - C_i\big), \\ c_{(i+1) \bmod M} &= H_n\big(m,\, I,\, L_0^{(i)},\, R_0^{(i)},\, L_1^{(i)}\big), \end{aligned}$$

with c_0 the value in the signature, and accept when the value written back into index 0 equals that c_0. The signing walk uses the same hash. It starts at π, and at that index it inserts the opened (L_0, R_0) and the transcript point L_1 in place of the three equations above.

References

[1] Yehuda Lindell. *Simple Three-Round Multiparty Schnorr Signing with Full Simulatability*. Cryptology ePrint Archive, Paper 2022/374. 2022. URL: https://eprint.iacr.org/2022/374.pdf.

[2] koe. *Mechanics of MobileCoin: First Edition*. Preview (10/11) v0.0.39. Apr. 2021. URL: https://cdn.prod.website-files.com/654151be5c8f07ebc15119fb/65de6a7179eab66b061930d8_Mechanics-of-MobileCoin-v0-0-39-preview-10-11.pdf.